Camera Calibration & AR

PRCV Spring 2026 · Project 4

Name: Varun Raghavendra
Submitted: March 27, 2026
Demo Video: https://drive.google.com/file/d/1jlCqBDTGTiq7BGDKWUlec5MgGzUPNV73/view?usp=sharing

Project Description

This project presents a camera calibration and augmented reality pipeline. The first stage detects a 9 × 6 internal-corner checkerboard pattern from a live webcam feed, collects sub-pixel refined corner observations from multiple views, and applies OpenCV's calibrateCamera to estimate the camera intrinsics, including focal lengths, principal point, and five distortion coefficients. These calibration parameters are then temporarily saved to a YAML file for later use. In the next stage, whenever the checkerboard is visible, solvePnP is used to estimate the board's full 6-DOF pose relative to the camera. Using this pose, projectPoints overlays 3D coordinate axes and the four outer corners of the checkerboard onto the image, while also rendering two stick-figure characters, a boy and a girl, that remain anchored to the board and move consistently with it in real time. As a final component, the project explores feature detection using the Harris corner detector as an example of a robust low-level vision method. An interactive threshold slider allows the user to vary the detection cutoff and directly observe how the number and spatial distribution of detected feature points change with the chosen parameter.

Task 1 — Detect and Extract Target Corners

The algorithm detects a 9×6 checkerboard target in each video frame by finding all 54 internal corner points and refining their locations to sub-pixel accuracy for reliable corner extraction. When detection succeeds, it overlays the corner pattern on the live video and reports the number of corners found along with the first corner's image coordinates.

Observed limitations: Detection fails at certain angles (eg.roughly over 60 degrees from front, parallel to the camera), because adjacent squares merge, and glare on paper printed out also causes some issues in detection of the checkerboard.

Task 2 — Select Calibration Images

The algorithm lets the user save only successful checkerboard detections for calibration, storing the detected 2D image corners from selected frames. For each saved frame, it creates the matching 3D checkerboard corner coordinates in a fixed world reference so image points and world points stay aligned for later camera calibration. I laid the printed checkerboard sheet on a stable and flat table with minimal background for easy detection.

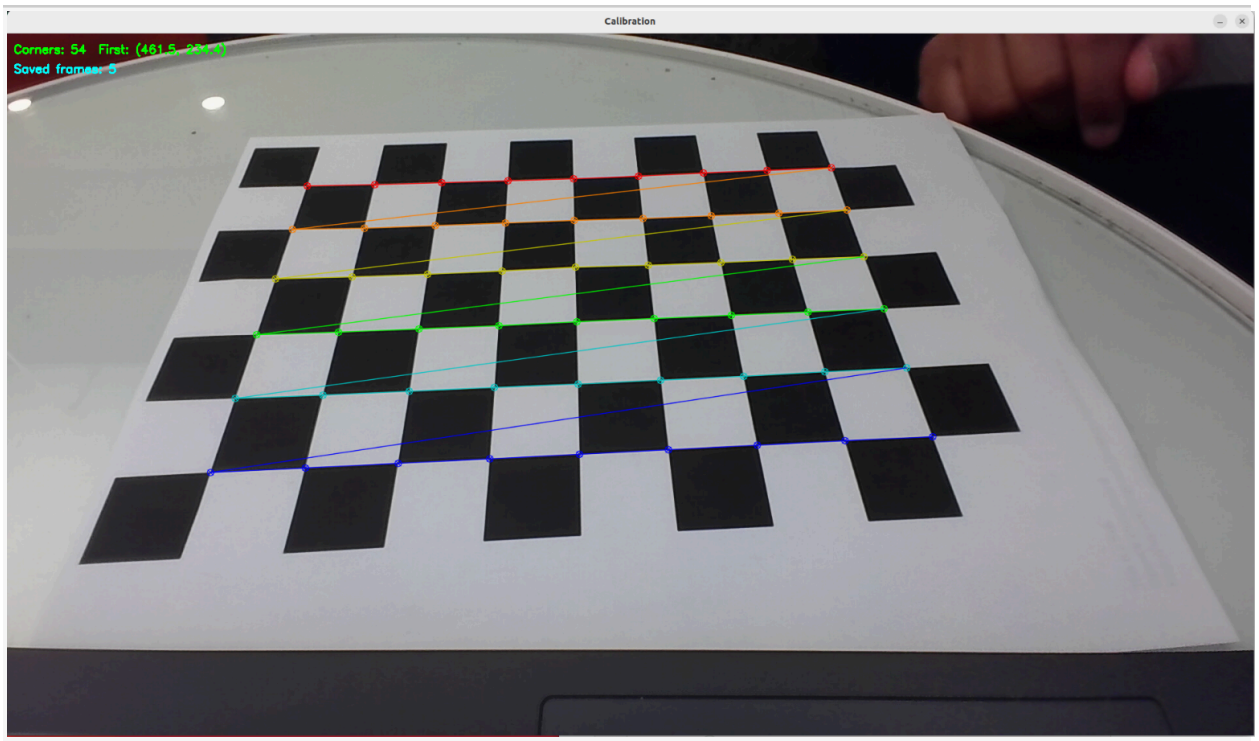

Figure 1. Snapshot of calibration frame with corners highlighted, collected from the camera viewpoint to improve the conditioning of the calibration.

Task 3 — Calibrate the Camera

The algorithm waits until at least five calibration views have been saved, then uses the known 3D checkerboard coordinates and their detected 2D image corners to estimate the camera's intrinsic parameters. Then we compute the camera matrix and distortion coefficients by minimizing reprojection error, while enforcing equal focal lengths under the square-pixel assumption. After calibration, we report the refined intrinsics and the final RMS reprojection error, which indicates how accurately the estimated camera model fits the observed corner measurements.

```
[s] Saved frame 1  (54 corners)
[s] Saved frame 2  (54 corners)
[s] Saved frame 3  (54 corners)
[s] Saved frame 4  (54 corners)
[s] Saved frame 5  (54 corners)

--- Before calibration ---
Camera matrix:
[1, 0, 960;
 0, 1, 540;
 0, 0, 1]
Distortion coefficients:
[0, 0, 0, 0, 0]

--- After calibration ---
RMS reprojection error: 0.487493 px
Camera matrix:
[1122.782462049727, 0, 954.0159248178805;
 0, 1122.782462049727, 528.7103578014027;
 0, 0, 1]
Distortion coefficients:
[0.02680134464669709, 0.04176750413266914, -0.00411668467960476, -0.00185061162266632, -0.1331816840830071]
```

Figure 2. Terminal output showing the camera matrix and distortion coefficients before and after calibration, plus the RMS reprojection error.

Task 4 — Calculate Current Position of the Camera

The algorithm reads the saved camera intrinsics (yaml file), detects the checkerboard in each frame, and uses the known 3D board geometry with the detected 2D corners to estimate the board pose relative to the camera. It outputs the rotation vector and translation vector in real time, where the rotation describes the board orientation and the translation gives its position in 3D space.

```
rvec=[0.07, 3.02, -0.84]  angle=179.57deg  tvec=[5.01, 2.50, 9.30]
rvec=[0.09, 3.01, -0.85]  angle=179.41deg  tvec=[5.06, 2.41, 9.34]
rvec=[0.11, 3.01, -0.86]  angle=179.46deg  tvec=[5.07, 2.23, 9.43]
rvec=[0.13, 3.00, -0.85]  angle=179.03deg  tvec=[5.11, 2.05, 9.55]
rvec=[0.15, 3.00, -0.85]  angle=179.08deg  tvec=[5.13, 1.90, 9.63]
rvec=[0.16, 3.00, -0.86]  angle=179.17deg  tvec=[5.07, 1.75, 9.70]
rvec=[0.17, 3.00, -0.85]  angle=178.91deg  tvec=[5.02, 1.63, 9.80]
rvec=[0.18, 3.00, -0.85]  angle=178.86deg  tvec=[4.99, 1.54, 9.84]
rvec=[0.17, 3.00, -0.85]  angle=179.08deg  tvec=[4.90, 1.44, 9.89]
rvec=[0.16, 3.00, -0.85]  angle=179.02deg  tvec=[4.82, 1.34, 9.96]
rvec=[0.15, 3.00, -0.85]  angle=179.13deg  tvec=[4.75, 1.23, 10.02]
rvec=[0.14, 3.00, -0.85]  angle=178.98deg  tvec=[4.72, 1.14, 10.10]
rvec=[0.15, 3.00, -0.85]  angle=178.93deg  tvec=[4.73, 1.03, 10.17]
rvec=[0.15, 3.00, -0.85]  angle=178.99deg  tvec=[4.73, 0.91, 10.23]
rvec=[0.15, 3.00, -0.85]  angle=178.90deg  tvec=[4.72, 0.78, 10.31]
rvec=[0.16, 3.00, -0.84]  angle=178.84deg  tvec=[4.72, 0.68, 10.38]
rvec=[0.16, 3.00, -0.85]  angle=178.98deg  tvec=[4.70, 0.57, 10.43]
rvec=[0.15, 3.00, -0.85]  angle=179.06deg  tvec=[4.65, 0.45, 10.50]
rvec=[0.15, 3.00, -0.85]  angle=178.99deg  tvec=[4.60, 0.31, 10.59]
```

Figure 3. For each frame, the code prints out the rotation and translation data in real time in the terminal. Here I am attaching the series of data when I move the checkerboard away from the camera in the y axis, hence the most linear change is visible in that axis

Task 5 — Project Outside Corners or 3-D Axes

The algorithm uses the estimated pose to project known 3D reference points from the checkerboard coordinate system back onto the live image in real time. It overlays the board boundary and a set of 3D coordinate axes so the projected points move consistently with the target as the camera or board changes position. This confirms that the recovered pose is geometrically correct, because the virtual corners and axes remain locked to the physical checkerboard in the video.

Do the reprojected points show up in the right places?

Yes when the checkerboard is in the frame, it's locked in.

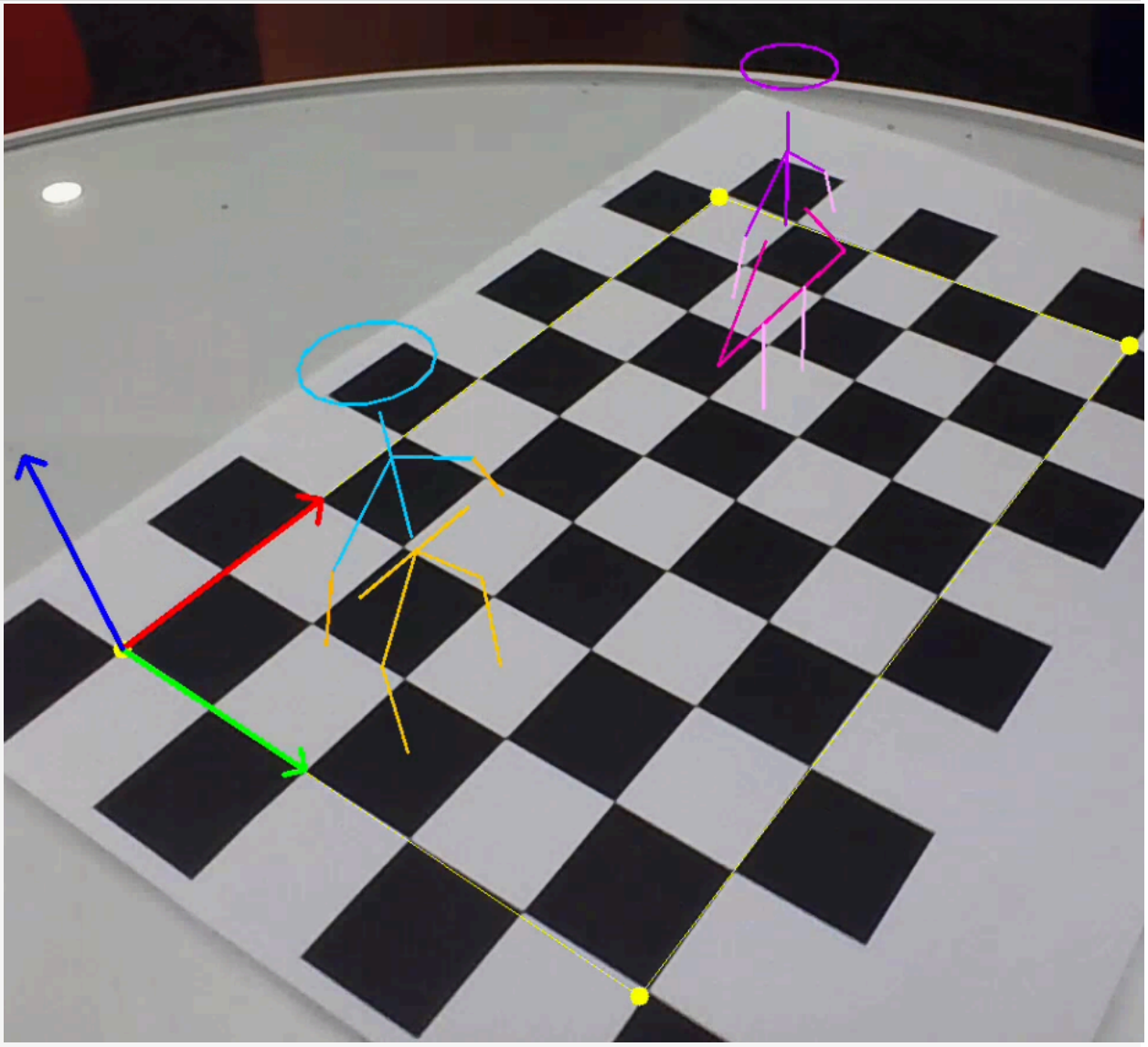

Figure 4. AR frame showing the four reprojected outer corners (cyan dots with border) and the three coordinate axes anchored to the board origin.

Task 6 — Create a Virtual Object

I tried to construct two virtual objects directly in the checkerboard's 3D world coordinate system (Stick-Figure Boy and Girl) and projected all of its line endpoints onto the image using the estimated pose. By drawing colored line segments between the projected points, it renders floating stick-figure shapes that appear anchored above the board rather than flat on the image. As the camera moves, the figures preserve their shape, depth, and orientation relative to the checkerboard, showing that the virtual object is correctly attached to the physical target in 3D space.

The demo video is attached in the beginning of the report

Task 7 — Detect Robust Features

The algorithm analyzes each frame to identify corner features, points where intensity changes sharply in multiple directions. A trackbar lets the user interactively adjust how sensitive the detection is, filtering out weaker responses. Detected corners are highlighted with red dots on the live feed, along with a running count displayed on screen.

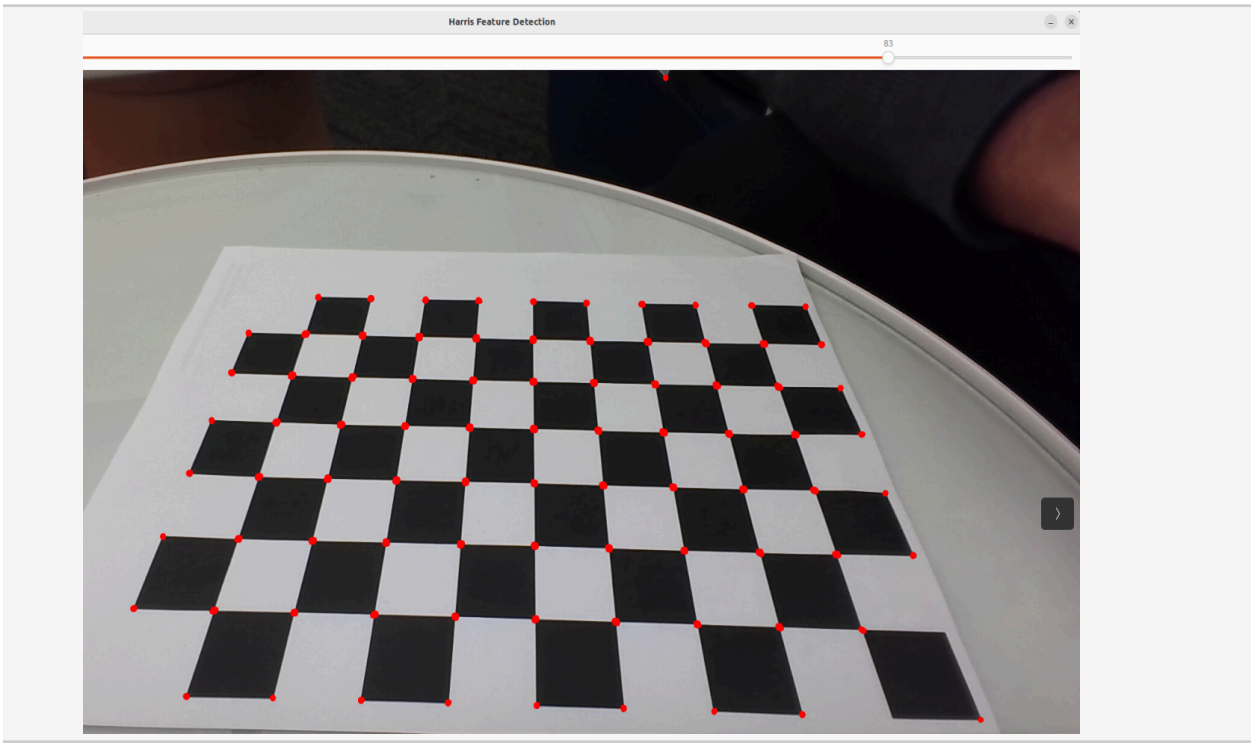

Figure 5. Harris corner detection at threshold=83% on a live webcam video stream. Red dots cluster at edges and high-contrast boundaries.

Reflections

I have been owning a Meta Quest 3 virtual reality headset for over a year now, and I was always fascinated by how objects in games like cricket and golf are rendered so naturally in space and how they stay aligned with the movement and pose of the VR controllers to create a realistic experience. This project helped me finally understand some of the core ideas behind that experience, especially camera calibration and feature detection. I realized that feature detection is one of the fundamental steps in spatial mapping, and although real VR headsets and robots use much more advanced and robust algorithms, the basic principles start here. Working on this project made the connection between course concepts and real-world applications much clearer for me, and it has made me even more interested in exploring mapping and localization in my final course project.

Acknowledgements

- PRCV Spring 2026 Class Material
- OpenCV Documentation — calibrateCamera, solvePnP, projectPoints, cornerHarris (corresponding pages under https://docs.opencv.org/3.4/)
- OpenCV Camera Calibration tutorial: https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html
- Camera Calibration in less than 5 Minutes with OpenCV Nicolai Nielsen **:** https://www.youtube.com/watch?v=_-BTKiamRTg
- Corner Detection | Edge Detection First Principles of Computer Vision : https://www.youtube.com/watch?v=Z_HwkG90Yvw
- Understanding Reprojection Error https://calib.io/blogs/knowledge-base/understanding-reprojection-errors?srsltid=AfmBOorQGDKMD-qiS9WfNSMf8HrzXKw7nSR2wpludyBtDAW6YViwrhoS
- https://stackoverflow.com/questions/43878684/interpreting-the-reprojection-error-from-camera-calibration
- LLM Assist : README.md, theoretical and code syntax understanding, error debugging, understanding the method of reprojecting outline of the board around identified corners, understanding the need for grayscale detection, understanding the need for thresholding, normalization and dilation for Harris Detection (before I was identifying many scattered points and somehow had to reduce them to just the checkerboard box corner detection).